



# 14.1. Bubble Sort & Comb Sort

[Previous Section:](#)

 [14. Sorting Algorithms](#)

**Contents:**

[1. Bubble Sort](#)

[2. Comb Sort](#)

[Summary](#)

[References](#)

Now that we understand **what sorting is** and **why it is important**, it's time to study some actual sorting algorithms.

We'll begin with four of the simplest and most famous sorting algorithms:

- Bubble Sort
- Comb Sort
- Selection Sort
- Insertion Sort

These algorithms are often called **basic sorting algorithms** because they are relatively easy to understand and implement.

That doesn't mean they are useless. Quite the opposite.

Although modern software often uses much more sophisticated algorithms, these basic algorithms teach us many fundamental ideas that appear again in advanced sorting techniques:

- Comparing elements

- Swapping values
- Finding the correct position
- Gradually building a sorted sequence

Think of these algorithms as learning how to walk before learning how to run.

Once we understand these techniques, understanding advanced algorithms such as Radix Sort, Merge Sort, and Quick Sort becomes much easier.

This section focuses on **Bubble Sort and Comb Sort**

## 1. Bubble Sort

Let's begin with perhaps the most famous sorting algorithm of all: **Bubble Sort**.

The name sounds a little strange. Why "Bubble"? Imagine bubbles rising through water. The largest bubbles naturally float upward until they reach the surface.

Bubble Sort behaves in almost the same way. During every iteration, the **largest unsorted element gradually "bubbles" toward the end of the array**.

The Main Idea: Suppose we have the following array.

|    |    |    |    |   |
|----|----|----|----|---|
| 12 | 45 | 20 | 11 | 9 |
|----|----|----|----|---|

Bubble Sort starts from the beginning of the array. It compares **two neighboring elements** at a time, `a[i]` and `a[i+1]`. If the first element is greater than the second, `a[i] > a[i+1]`, then the two elements are swapped.

Otherwise, they remain in place. The algorithm continues moving forward until it reaches the end of the array.

By the end of one complete traversal, something interesting happens. The **largest value** has been pushed all the way to the end of the array. Just like a bubble rising to the surface.

- **First Iteration** - Let's watch the process.

Initial array:

|    |    |    |    |   |
|----|----|----|----|---|
| 12 | 45 | 20 | 11 | 9 |
|----|----|----|----|---|

$12 < 45 \rightarrow$  No swap.

|    |    |    |    |   |
|----|----|----|----|---|
| 12 | 45 | 20 | 11 | 9 |
|----|----|----|----|---|

$45 > 20 \rightarrow$  Swap.

|    |    |    |    |   |
|----|----|----|----|---|
| 12 | 20 | 45 | 11 | 9 |
|----|----|----|----|---|

$45 > 11 \rightarrow$  Swap.

|    |    |    |    |   |
|----|----|----|----|---|
| 12 | 20 | 11 | 45 | 9 |
|----|----|----|----|---|

$45 > 9 \rightarrow$  Swap.

|    |    |    |   |    |
|----|----|----|---|----|
| 12 | 20 | 11 | 9 | 45 |
|----|----|----|---|----|

Notice something? The largest value **45** has reached its final position. We never need to compare it again.

- Second Iteration: Now we ignore the last element. Only the remaining unsorted portion is processed.

|    |    |    |   |    |
|----|----|----|---|----|
| 12 | 20 | 11 | 9 | 45 |
|----|----|----|---|----|

$12 < 20 \rightarrow$  No Swap.

|    |    |    |   |    |
|----|----|----|---|----|
| 12 | 20 | 11 | 9 | 45 |
|----|----|----|---|----|

20 > 11 → Swap.

|    |    |    |   |    |
|----|----|----|---|----|
| 12 | 11 | 20 | 9 | 45 |
|----|----|----|---|----|

20 > 9 → Swap.

|    |    |   |    |    |
|----|----|---|----|----|
| 12 | 11 | 9 | 20 | 45 |
|----|----|---|----|----|

Two elements are sorted

|    |    |   |    |    |
|----|----|---|----|----|
| 12 | 11 | 9 | 20 | 45 |
|----|----|---|----|----|

At the end of the second iteration, the **second-largest element (20)** has reached its final position, so the last **two** elements are now sorted.

- Eventually, we will reach the iteration where not a single swap occurs. At that point, the algorithm knows the array is sorted.

|   |    |    |    |    |
|---|----|----|----|----|
| 9 | 11 | 12 | 20 | 45 |
|---|----|----|----|----|

- **Number of Comparisons:** One interesting property of Bubble Sort is that the number of comparisons decreases after every iteration.

Why? Because every complete traversal places one additional largest element into its final position. Those elements never move again.

Suppose the array contains **n** elements.

| Iteration | Comparisons   |
|-----------|---------------|
| 0         | $n - 1$       |
| 1         | $n - 2$       |
| 2         | $n - 3$       |
| ...       | ...           |
| $i$       | $n - (i + 1)$ |

Adding them together gives  $[(n - 1) + (n - 2) + \dots + 1 = \frac{n(n-1)}{2} \rightarrow$   
Therefore: **Time Complexity**  $O(n^2)$

- Can Bubble Sort Move in Reverse? Yes. So far, we've started from the beginning and pushed the **largest element** toward the end. But we can also reverse the direction.
  - Instead of moving left  $\rightarrow$  right, we move right  $\rightarrow$  left.
  - In this version, the **smallest element** gradually moves toward the beginning.

Before:

|    |    |    |    |   |
|----|----|----|----|---|
| 45 | 20 | 11 | 12 | 9 |
|----|----|----|----|---|

45 20 11 12 9

After one iteration:

|   |    |    |    |    |
|---|----|----|----|----|
| 9 | 45 | 20 | 11 | 12 |
|---|----|----|----|----|

The overall idea remains exactly the same. Only the traversal direction changes.

### • Complexity

| Complexity       | Value                      |
|------------------|----------------------------|
| Best Case        | $O(n)$ (optimized version) |
| Average Case     | $O(n^2)$                   |
| Worst Case       | $O(n^2)$                   |
| Space Complexity | $O(1)$                     |

### • Implementation

```
def bubble_sort(a): # Bubble Sort [Time(n), Space(n) = 0(n
^ 2), 0(1)]
    n = len(a)
```

```

for i in range(1, n, 1): # Iterate through array
    is_sorted = True
    # Compare all couple of nearest numbers
    # The lower number is rotated to the left side of the array
    for j in range(0, n - i, 1):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
            is_sorted = False # The array is not yet sorted
    if is_sorted: # If the array is already sorted
        break
return a

```

Bubble Sort is extremely easy to implement, but it becomes inefficient for large datasets because it performs many unnecessary comparisons and swaps.

## 2. Comb Sort

Bubble Sort is simple. But it also has one major weakness. It only compares **adjacent elements**. This means that if an element is very far from its correct position, it can only move **one position per comparison**.

Consider this array.

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 12 | 45 | 20 | 11 | 9 | 21 | 28 | 14 |
|----|----|----|----|---|----|----|----|

Notice the value **9**. It belongs near the beginning, yet it is currently located almost at the end of the array.

Using Bubble Sort, the value **9** can only move left **one position at a time**. It may require several complete iterations before finally reaching its correct location.

Can we move it faster? **Comb Sort answers "Yes."**

The Main Idea: Instead of comparing neighboring elements (Gap = 1), Comb Sort starts by comparing elements that are **much farther apart**. Instead of

comparing 12 and 45 (only compare adjacent element), we compare 12 and 9 if the gap is four (*compare elements that are four other elements away*).

By allowing large jumps during the early stages, elements can travel toward their correct positions much more quickly.

Let's sort the following array.

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 12 | 45 | 20 | 11 | 9 | 21 | 28 | 14 |
|----|----|----|----|---|----|----|----|

- Pass 1 — Gap = 4: The algorithm compares elements that are **4 positions apart**. Perform the comparisons one by one.

12 > 9 → Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 45 | 20 | 11 | 12 | 21 | 28 | 14 |
|---|----|----|----|----|----|----|----|

45 > 21 → Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 21 | 20 | 11 | 12 | 45 | 28 | 14 |
|---|----|----|----|----|----|----|----|

20 < 28 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 21 | 20 | 11 | 12 | 45 | 28 | 14 |
|---|----|----|----|----|----|----|----|

11 < 14 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 21 | 20 | 11 | 12 | 45 | 28 | 14 |
|---|----|----|----|----|----|----|----|

After the first pass, the array becomes

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 21 | 20 | 11 | 12 | 45 | 28 | 14 |
|---|----|----|----|----|----|----|----|

Notice something interesting. The value **9** is moved directly from index **4** to index **0** in a single comparison. Bubble Sort would have needed several passes to accomplish the same thing.

- Pass 2 — Gap = 3: Now we reduce the gap to **3**. The comparisons are performed as follows:

9 < 11 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 21 | 20 | 11 | 12 | 45 | 28 | 14 |
|---|----|----|----|----|----|----|----|

21 > 12 → Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 12 | 20 | 11 | 21 | 45 | 28 | 14 |
|---|----|----|----|----|----|----|----|

20 < 45 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 12 | 20 | 11 | 21 | 45 | 28 | 14 |
|---|----|----|----|----|----|----|----|

11 < 28 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 12 | 20 | 11 | 21 | 45 | 28 | 14 |
|---|----|----|----|----|----|----|----|

21 > 14 → Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 12 | 20 | 11 | 14 | 45 | 28 | 21 |
|---|----|----|----|----|----|----|----|

The array is already becoming much more organized.

- Pass 3 — Gap = 2: Now we compare elements separated by **2 positions**.

9 < 20 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 12 | 20 | 11 | 14 | 45 | 28 | 21 |
|---|----|----|----|----|----|----|----|



12 > 11 → Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 20 | 12 | 14 | 45 | 28 | 21 |
|---|----|----|----|----|----|----|----|

20 > 14 → Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 14 | 12 | 20 | 45 | 28 | 21 |
|---|----|----|----|----|----|----|----|

12 < 45 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 14 | 12 | 20 | 45 | 28 | 21 |
|---|----|----|----|----|----|----|----|

20 < 28 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 14 | 12 | 20 | 45 | 28 | 21 |
|---|----|----|----|----|----|----|----|

45 > 21 → Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 14 | 12 | 20 | 21 | 28 | 45 |
|---|----|----|----|----|----|----|----|

After swapping, only a few elements remain out of place.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 14 | 12 | 20 | 21 | 28 | 45 |
|---|----|----|----|----|----|----|----|

- Pass 4 — Gap = 1: Eventually, the gap reaches **1**. Now Comb Sort behaves exactly like Bubble Sort.

9 < 11 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 14 | 12 | 20 | 21 | 28 | 45 |
|---|----|----|----|----|----|----|----|

11 < 14 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 14 | 12 | 20 | 21 | 28 | 45 |
|---|----|----|----|----|----|----|----|

14 > 12 → Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 12 | 14 | 20 | 21 | 28 | 45 |
|---|----|----|----|----|----|----|----|

14 < 20 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 12 | 14 | 20 | 21 | 28 | 45 |
|---|----|----|----|----|----|----|----|

20 < 21 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 12 | 14 | 20 | 21 | 28 | 45 |
|---|----|----|----|----|----|----|----|

21 < 28 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 12 | 14 | 20 | 21 | 28 | 45 |
|---|----|----|----|----|----|----|----|

28 < 45 → No Swap.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 12 | 14 | 20 | 21 | 28 | 45 |
|---|----|----|----|----|----|----|----|

After one or more Bubble Sort passes, no more swaps are performed. The array is at which completely sorted.

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 12 | 14 | 20 | 21 | 28 | 45 |
|---|----|----|----|----|----|----|----|

- **How Do We Choose the Next Gap?** You might now wonder, **why did we choose 4, then 3, then 2, and finally 1? Meaning we could simply subtract one every iteration?** Yes, that was the point of the demonstration.

However, experiments have shown that reducing the gap using a constant ratio generally produces better performance.

Comb Sort therefore introduces a value called the **shrink factor**. The most commonly used shrink factor is 1.3. Instead of subtracting one, we repeatedly divide the current gap by the shrink factor →  $\text{gap} = \text{gap} / 1.3$

For an array of length 8, Initial gap = 8

- $8 / 1.3 \approx 6$
- $6 / 1.3 \approx 4$
- $4 / 1.3 \approx 3$

- $3 / 1.3 \approx 2$
- $2 / 1.3 \approx 1$

Once the gap becomes **1**, it remains **1**, and the algorithm finishes exactly like Bubble Sort. The shrink factor is therefore what gradually transforms Comb Sort into Bubble Sort.

- **Why Is Comb Sort Faster?** Bubble Sort only allows elements to move one position at a time.

Comb Sort allows elements to move several positions during the early passes, correcting large mistakes much sooner.

By the time the gap reaches **1**, most elements are already close to their correct positions. Bubble Sort only needs to perform the final cleanup.

In other words, **Comb Sort gives Bubble Sort a head start.**

- Complexity

| Complexity       | Value                     |
|------------------|---------------------------|
| Best Case        | $O(n \log n)$ (practical) |
| Average Case     | $O(n^2)$                  |
| Worst Case       | $O(n^2)$                  |
| Space Complexity | $O(1)$                    |

- Implementation

```
def comb_sort(a): # Comb Sort [Time(n), Space(n) = O(n ^
2), O(1)]
    n = len(a)
    gap = n
    is_sorted = False
    # If gap == 1 and the array is not sorted
    # The next iteration performs like Bubble Sort
    while (gap != 1) or (not is_sorted):
        gap = int(gap / 1.3) # Shrink Factor = 1.3
        if gap <= 1: # If gap is less than 1, treat it as
gap = 1
```

```

        gap = 1
    # Use Bubble Sort for values that have gap
    is_sorted = True
    for i in range(0, n - gap, 1):
        if a[i] > a[i + gap]:
            a[i], a[i + gap] = a[i + gap], a[i]
            is_sorted = False # The array is not yet s
orted
    return a

```

## Summary

Bubble Sort and Comb Sort are both **comparison-based sorting algorithms** that repeatedly compare pairs of elements and swap them when they are in the wrong order.

- Bubble Sort compares **adjacent elements**, causing the largest unsorted value to gradually "bubble" toward the end of the array after each iteration. While simple and easy to understand, it becomes inefficient for large datasets because elements can only move one position at a time.
- Comb Sort improves upon Bubble Sort by comparing elements separated by a larger **gap** during the early passes. As the gap gradually shrinks to **1**, the algorithm eventually behaves like Bubble Sort, but with most misplaced elements already much closer to their correct positions. This simple improvement often makes Comb Sort significantly faster in practice.

Having explored algorithms that rely primarily on repeated swapping, we will now study two different approaches to sorting: **Selection Sort**, which repeatedly selects the next smallest element, and **Insertion Sort**, which gradually builds a sorted portion of the array one element at a time.

## References

[https://drive.google.com/file/d/1jK22rfWszKKgPgeprquGoeJKqgBg8rUY/view?usp=drive\\_link](https://drive.google.com/file/d/1jK22rfWszKKgPgeprquGoeJKqgBg8rUY/view?usp=drive_link)

<https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm/>

<https://www.geeksforgeeks.org/dsa/comb-sort/>

**Next Section:**

 [14.2. Selection Sort & Insertion Sort](#)